

Data Engineering Project Requirements

2-3 weeks, until end of August

1. Objective and Scope

This document defines what every student project must include, regardless of which dataset is chosen, and then specifies the custom operations required for each of the three approved datasets. Every project uses Apache Airflow (orchestration), DuckDB (local warehouse), and dbt (transformation and testing), and every project runs over roughly 3 weeks.

Structure of this document: Section 2 covers requirements that apply to every project no matter the dataset. Section 3 covers the dataset-specific operations: pick exactly one of the three.

1.1 Dataset Options

Choose one:

Dataset	Source	Difficulty	Primary Challenge
Chicago 311	data.cityofchicago.org	Moderate	Incremental API extraction, upserts, messy categorical data, star schema
IMDb	datasets.imdbws.com	Moderate	Large multi-table files, true star schema, many-to-many bridges
Divvy Bikeshare	divvybikes.com/system-data	Moderate	Slowly-changing station dimension, monthly incremental loads, trip-level data quality

1.2 Source Links

Chicago 311 Service Requests: https://data.cityofchicago.org/Service-Requests/311-Service-Requests/v6vf-nfxy/about_data

IMDb bulk data: <https://datasets.imdbws.com/>

Divvy Bikeshare Trip Data: <https://divvybikes.com/system-data>

2. Common Requirements: Apply to Every Project

These requirements are identical regardless of which dataset is chosen. They exist so that every submission is comparable and gradable on the same basis.

Git repository present for this project.

2.1 Project Structure

Every project uses this folder layout (rename the root folder for the chosen dataset):

```
project_name/
├── dags/
│   └── ingest_dag.py           # Airflow DAG: extract, convert, load
├── dbt_project/
│   ├── models/
│   │   ├── staging/
│   │   └── intermediate/     # only if a reshaping step is needed
```

├── marts/	
│ └── sources.yml	
├── macros/	# if any custom macros are used
├── tests/	
└── dbt_project.yml	
raw/	# landed parquet files (git-ignored)
warehouse.duckdb	# the DuckDB database file
docs/	
│ └── project_documentation.docx	# the final written report (Section 2.4)
└── README.md	

2.2 dbt Modeling Standard

- Staging models (stg_*): one per raw source, no business logic, only renaming, type casting, and null handling.
- Intermediate models: only used if the dataset needs a reshaping step (e.g., a pivot) between staging and marts.
- Marts: the final, query-ready tables that answer the project's deliverable questions.
- Every fact table's grain (what one row represents) must be explicitly stated in the documentation.
- Every project must implement at least one slowly changing dimension (SCD Type 2), built with a dbt snapshot. Identify which dimension in the chosen dataset genuinely changes over time, and document why that change matters (see Section 3 for the specific dimension to use per dataset).

2.3 Testing Minimums

- At least 5 dbt tests across the project (not_null, unique, relationships, and at least one custom SQL test).
- Every primary key tested for uniqueness and non-null.
- At least one test that encodes a real domain rule for the chosen dataset (e.g., a value that must fall within a valid range), not just generic key tests.
- Tests that are expected to legitimately fail on some rows (known dirty data) should use severity: warn with a one-line justification in the documentation, not be silently dropped.

2.4 Airflow Pipeline Standard

- The DAG must contain at least four logical stages that include extraction, loading, transformation, and validation.
- Task dependencies must be explicitly defined and visible in the Airflow Graph View.
- Failed tasks must be configured with at least one retry attempt.
- All tasks must produce meaningful log output that can be used for troubleshooting.
- The DAG must be idempotent: re-running the same execution should not create duplicate records.

- Incremental processing should be used whenever supported by the source system or dataset design.
- External systems, credentials, or API tokens (if they exist) must be managed through Airflow Connections or configuration files rather than hard-coded values.
- The pipeline must fail if critical transformation or data quality validation steps fail.

2.5 Required Project Documentation

A working pipeline alone is not a complete deliverable. Every project must include a written report (recommended: a Word document, 6-10 pages), graded with equal weight to the working code, covering:

- **Executive Summary:** what the project does and what question it ultimately answers, in under one paragraph.
- **Dataset Description:** source, size, update frequency, and any scoping decisions made (what subset was extracted and why).
- **Architecture Overview:** a diagram or clear description of how Airflow, DuckDB, and dbt connect in this project, including schedule and trigger sequence.
- **Data Modeling Explanation:** the entities chosen, the grain of each table, why that grain was chosen, and the relationships between tables.
- **Project Structure Walkthrough:** the folder layout with a one-line explanation of what lives in each folder and why.
- **Airflow DAG Explanation:** a screenshot of the DAG graph view, plus a written explanation of each task, its dependencies, and the schedule.
- **dbt Lineage and Tests:** a screenshot of the dbt docs lineage graph, a list of tests implemented, and why each one exists.
- **How to Run:** step-by-step instructions to reproduce the pipeline from a clean DuckDB file.
- **Answers to the Deliverable Questions:** each question restated, the model/query used, the result, and one sentence of interpretation.
- **Challenges and Lessons Learned:** at least three specific problems encountered and how each was resolved.
- Screenshots must be real captures from the student's own Airflow UI and dbt docs site, not stock images.
- Save the documentation at docs/project_documentation.docx per the folder structure in 2.1.

2.6 Rough Pacing (any dataset)

Roughly one week per phase is a reasonable default, though the specific operations in Section 3 determine how the time actually splits: Week 1, extraction/ingestion into DuckDB; Week 2, modeling and testing; Week 3, deliverable questions, documentation, and polish.

2.7 Final Success Criteria Checklist

- Airflow DAG runs end-to-end without manual intervention and appears correctly in the UI.

- Raw data lands as Parquet, not left as CSV/JSON.
- dbt project follows the staging → (intermediate) → marts structure.
- Testing minimums from 2.3 are met.
- dbt docs generate runs cleanly and produces a browsable lineage graph.
- At least one slowly changing dimension (SCD Type 2) is implemented via a dbt snapshot, with the dimension's change-over-time behavior documented.
- All deliverable questions (Section 3) are answered with a model or query the reader can find and re-run.
- Full written documentation exists covering every item in 2.5.
- A README exists explaining how to run the whole pipeline from a clean DuckDB file.

3. Dataset-Specific Custom Operations

Complete only the subsection for the dataset chosen. These are the operations that go beyond the common requirements in Section 2, the parts that are genuinely different depending on the data.

3.1 Chicago 311

Custom operations required

- Extract via the Socrata API using `$where=created_date > :last_run_date` for true incremental extraction, not a full re-download each run.
- Store the `last_run_date` watermark in a DuckDB control table (not an Airflow Variable).
- Register a free Socrata app token as an Airflow Connection to avoid rate limiting.
- Normalize inconsistent text fields (`sr_type`, `sr_short_code`, `status`) in staging: trim/standardize casing and map known variants to canonical values.
- Build an incremental dbt model keyed on `sr_number` using `is_incremental()` with a merge strategy, so status changes (Open → Completed) overwrite the old row instead of duplicating it.
- Add a dbt snapshot on the full request table to capture every status transition over time.
- Required SCD Type 2: build `dim_ward` as a dbt snapshot. Chicago redrew its ward boundaries in 2023, so a request's ward must reflect the map in effect when the request was made, not the current one. Seed the boundary-change date as a small reference table and snapshot the ward assignment logic against it, then join `fct_service_requests` to the ward version valid at `created_date`.
- Custom tests: `closed_date >= created_date`; latitude/longitude fall within Chicago's bounding box; `not_null` on `community_area` with severity: warn.

Deliverable questions

- Which community areas have the slowest median resolution time for pothole/graffiti requests, and has it improved or worsened year-over-year?
- Which request types have the highest ratio of reopened/duplicate complaints (same location, same type, within a short window)?

- Do ward-level request volumes look different when requests are attributed to the ward in effect at the time versus the current ward map, for the years spanning the 2023 redistricting?

End goal

A daily-refreshed mart that reliably reflects the current status of every open request (no duplicates from re-pulls), a full history of status changes, and correct historical ward attribution across the 2023 boundary change, answering: where is the city falling behind on response times, and is it getting better or worse?

3.2 IMDb

Custom operations required

- Ingest title.basics, title.ratings, title.crew, title.principals, name.basics, and title.akas as separate staging models.
- Split the pipe-delimited genres column into a bridge table (stg_title_genres), a many-to-many relationship.
- Build a true star schema: fct_title_ratings (grain: one row per title) with dimensions dim_title, dim_genre, dim_person, dim_year.
- Build bridge_title_crew (title ↔ person ↔ role: director/writer/actor) from title.crew and title.principals.
- Build dim_person from name.basics, exploding the knownForTitles column.
- Add a dbt snapshot on title.ratings so rating changes are tracked across successive loads (this tracks a changing measure, not a dimension).
- Required SCD Type 2: build dim_title as a dbt snapshot on its core descriptive attributes (primaryTitle, genres, runtimeMinutes). IMDb periodically corrects or reclassifies this metadata between weekly loads, and marts should be able to reflect what a title's metadata looked like at the time of a given load, not silently overwrite it.
- Custom tests: averageRating between 0-10; startYear not in the future; relationships test between the fact table and every dimension.

Deliverable questions

- Who are the top 10 directors by average rating, restricted to those with 5+ titles and 1,000+ combined votes?
- Has the average movie runtime changed by decade, and does runtime correlate with rating?
- Which genres have the highest ratio of “hidden gems” (high rating, low vote count) versus “overrated” titles (low rating, high vote count)?

End goal

A queryable star schema where a single dbt mart can answer: who are the most consistently well-rated directors/actors of the last 20 years, and in which genres do they concentrate, with every underlying join tested and documented.

3.3 Divvy Bikeshare

Custom operations required

- Download monthly trip files (CSV, zipped) from Divvy's public system-data bucket via an Airflow task; land each month as its own Parquet file in raw/, partitioned by year-month.
- Required SCD Type 2: build a dbt snapshot on the station dimension (station_id, station_name, latitude, longitude) to track renames, relocations, and re-issued IDs over time. Divvy's own station metadata is not stable release over release, which is the core particularity of this dataset.
- Write a staging model that reconciles station identity across months where the same physical station appears under a different name or ID; use a seed file as a manual mapping table if automatic reconciliation isn't reliable enough.
- Build fct_trips at one-row-per-ride grain with two foreign keys into the station snapshot (start station and end station): the two-FK-to-one-dimension pattern, but time-bound this time, since a trip must join to the station record valid at the time of the ride, not the current one.
- Make the trip load incremental, keyed on ride_id, so re-running a month's ingestion doesn't duplicate rows.
- Compute trip duration in dbt from started_at/ended_at directly rather than trusting any provided duration field, and flag anomalies (negative or implausibly long durations).
- Custom tests: trip duration > 0; rideable_type within accepted_values (classic_bike, electric_bike, docked_bike); not_null on start_station_id with a documented severity: warn for legitimately dockless/lost-bike trips.

Deliverable questions

- Which stations have the largest net imbalance between departures and arrivals on an average weekday (useful for rebalancing), and does this differ between members and casual riders?
- How have average trip duration and the member/casual split changed month over month across the available history?
- Which stations were renamed, relocated, or re-issued a new ID over the available history, and how would trip counts be miscounted if that weren't corrected for?

End goal

A trips fact table joined to a properly time-versioned station dimension, so the student can pick any month and correctly attribute trips to the right station identity even across renames or relocations, answering where bikes pile up, where they run out, and how usage shifts over time.